

SYSTEM DESIGN

Chapter 54: Design Uber / Lyft

Personal Notes

The previous three designs (URL shortener, news feed, chat) each introduced new patterns: stateless request-response, fan-out, and stateful WebSocket. Uber introduces something entirely new — SPATIAL INDEXING. "Find nearby drivers" isn't a normal database query. You can't just `SELECT * FROM drivers WHERE distance < 5km` — that's $O(n)$ per request, prohibitive at Uber's scale (millions of drivers, thousands of requests per second).

This chapter designs a ride-hailing system focused on the CORE challenge: real-time driver-rider matching using spatial data structures. Along the way we'll cover geohashing (the workhorse of location systems), quadtrees (the alternative), the trip lifecycle from request to completion, ETA calculation via road networks, and surge pricing. Beyond ride-hailing, these patterns power food delivery, dating apps, real-estate search — anything with "nearby X" queries.

1. The Problem

Design a ride-hailing service. Riders open the app and see nearby drivers on a map. When they request a ride, the system finds a nearby driver, assigns them the trip, and tracks the journey to completion. Drivers see incoming ride requests, accept or decline, and navigate to pickup and drop-off.

Rider opens app → sees nearby drivers (updates every 4 seconds)
Rider requests ride → matched to nearest available driver
Driver accepts → rider sees driver's location + ETA
Driver picks up → trip begins, tracked in real-time
Driver drops off → trip complete, fare calculated, both rated

Core challenges:

1. Track 5M+ moving driver locations, updating every 4 sec
2. Efficiently find "nearest 10 drivers" to any point
3. Match driver-rider fairly + fast (< 3 sec)
4. Track ongoing trips with real-time driver location

2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- Drivers report location every ~4 seconds while online
- Riders see nearby drivers on the map (visualization)
- Rider requests a ride → system matches to a nearby driver
- Driver accepts / declines → alternative matching if declined

- Trip tracking (real-time location during ride)
- Fare calculation, payment, rating after trip
- ETA calculation (arrival time, remaining trip time)

2.2 Non-Functional Requirements

- Scale — 100M users, 5M active drivers, 1M active trips at peak
- Latency — matching decision in < 3 seconds; location updates propagate in < 1 sec
- Availability — 99.99%+ uptime (downtime blocks all ride requests)
- Consistency — strong for trips (assignment, payment); eventual for nearby-driver visualization

2.3 Constraints to Confirm

- Location update frequency — 4 sec is standard; higher rate = more infra load
- Match radius — 5 km typical; larger = more choices but longer ETA
- Multi-region — is this global, or per-city deployment?
- Ride types — economy, XL, luxury? (Affects matching filters)

The critical clarification: "How fresh does the 'nearby drivers' visualization need to be?" Riders expect drivers to move smoothly on the map, but a 3-4 second lag is acceptable. This lets us use EVENTUAL consistency for driver positions on the map — huge scaling win. Matching requires strongly-consistent state, but visualization doesn't.

3. Step 2 — Capacity Estimation

Location Updates

5M active drivers × (1 update / 4 seconds)
= 1.25M location updates/sec

Peak (rush hour, urban areas): × 2 = 2.5M updates/sec

This is the dominant write workload. Every driver, every 4 seconds, even when idle. Not batched — must be near-real-time.

Rider Queries

100M users × 10% active in peak hour × 3 map opens/session
≈ 30M rider map queries per hour
÷ 3,600 seconds/hour
≈ 8,300 queries/sec baseline
× 3 peak
≈ 25,000 rider queries/sec

Each query returns ~10-20 nearby drivers.

Ride Requests / Trips

1M active trips at any given moment (peak)
Average trip: 20 minutes
→ ~3M new trip requests per hour = ~850/sec (avg), ~2,500/sec peak

Small compared to location updates – but each trip is high-value (money, ratings, safety) and must be strongly consistent.

Storage

Live driver locations: 5M rows in-memory (~1 GB)
Trip history: 3M trips/hour × 24 × 365 = ~25B trips/year
× 500 B = ~12 TB/year of trip records
Location breadcrumbs (for trip playback): ~100 KB per trip
→ 300 GB/hour of breadcrumb data (peak)

Live state in Redis. Historical in Cassandra + cold storage for breadcrumbs.

4. Step 3 — API Design

Driver-side (mostly WebSocket for real-time)

WebSocket events (driver ↔ server):

- { "type": "location_update", "lat": 37.77, "lng": -122.42, "heading": 45 }
- ← { "type": "ride_request", "trip_id": "...", "pickup": {...}, "expires_in": 15 }
- { "type": "accept_ride", "trip_id": "..." }
- { "type": "trip_status", "trip_id": "...", "status": "arrived" | "started" | "completed" }

HTTP:

POST /api/driver/online	Go online (available for rides)
POST /api/driver/offline	Go offline

Rider-side

GET /api/nearby-drivers?lat=...&lng=...&radius=5km
→ { "drivers": [{ "id": ..., "lat": ..., "lng": ..., "eta_min": 3 }, ...] }

POST /api/trips/request
{ "pickup": {...}, "destination": {...}, "ride_type": "economy" }
→ { "trip_id": "...", "driver": {...}, "eta_pickup_min": 4 }

GET /api/trips/:trip_id Get current trip status
POST /api/trips/:trip_id/cancel Cancel active trip

```
WebSocket:
  ← { "type": "driver_moved", "lat": ..., "lng": ... }           // during
ride
  ← { "type": "trip_status", "status": "arrived" | ... }
```

Push vs poll for rider-side driver updates: Riders viewing the map WHILE waiting need frequent driver location updates. HTTP polling every 4 seconds works, but WebSocket push is smoother and lower-load. Use WebSocket for authenticated riders in an active trip; polling for casual browsing.

5. Step 4 — Data Model

```
Table: drivers
  driver_id (PK) | name | phone | vehicle_info | status | rating

Table: driver_locations (Redis – hot)
  Key: driver:{driver_id}
  Value: { lat, lng, heading, speed, last_updated, status }
  TTL: 60 seconds (auto-clean if driver disappears)

Table: trips
  trip_id (PK) | rider_id | driver_id | status | pickup | destination
              | requested_at | started_at | completed_at | fare
  Sharded by trip_id.

Table: trip_events (Cassandra – trip lifecycle log)
  trip_id | event_seq | event_type | timestamp | metadata
  – Every state change (requested, matched, arrived, started,
  completed).

Table: trip_breadcrumbs (Cassandra – driver's path during trip)
  trip_id | timestamp | lat | lng
  – For post-trip playback, disputes, analytics.

Spatial Index (Redis GEO or custom):
  geohash → set of driver_ids currently at that geohash
  – Enables "find drivers near this lat/lng" in  $O(\log n)$ .
```

6. The Core Challenge — Finding Nearby Drivers

This is the design's beating heart. Given a rider's location (lat, lng), find the 10-20 nearest available drivers. Naïve approach: check distance to every driver — $O(N)$ with 5M drivers = ~5M operations per query. At 25K queries/sec, that's 125 BILLION operations/sec. Impossible.

We need a SPATIAL INDEX that groups nearby drivers together, so "find nearby" is $O(\log n)$ or better. Three main options: GEOHASHING, QUADTREES, and R-TREES. Geohashing is the industry favorite for real-time systems like Uber.

7. Geohashing — The Workhorse

GEOHASHING encodes a lat/lng pair into a SHORT STRING. Nearby locations get similar-looking strings (share long common prefixes). "Find drivers near X" becomes "find drivers whose geohash starts with the same prefix as X" — a fast string range query.

How Geohashing Works

Divide the world into a 2×4 grid:
(roughly)

d	e	f	g	← northern strip
s	t	u	v	← lower strip (san francisco → 9)
...				

Recursively subdivide each cell into a 2×4 subgrid.
Each level adds one base-32 character to the hash.

Precision by length:

1 char	= ~5000 km cell	(country-scale)
4 char	= ~40 km cell	(metro area)
6 char	= ~1.2 km cell	(neighborhood) ← useful for ride matching
8 char	= ~40 m cell	(city block)
12 char	= ~4 cm cell	(survey-grade)

Example: San Francisco → "9q8yyz"
 LA → "9q5ct"

They share "9q" prefix → both in Western US, but different regions.

Using Geohashes for Nearby Search

```
// Driver reports location:
String hash = geohash(37.77, -122.42, 6); // "9q8yyz"
redis.set("driver:D42", { location, hash });
redis.sadd("geo:9q8yyz", "D42");          // add to geohash's set

// Rider searches:
String riderHash = geohash(riderLat, riderLng, 6);
Set<String> drivers = redis.smembers("geo:" + riderHash);
// Also check 8 neighbor cells to handle edge cases
for (String neighbor : neighbors(riderHash)) {
    drivers.addAll(redis.smembers("geo:" + neighbor));
}
// Return top 20 closest by Haversine distance
return topK(drivers, 20);
```

The border problem: A rider on the EDGE of a geohash cell might have the nearest driver in a NEIGHBOR cell. Always query the 8 surrounding cells too. Without this, riders 10 meters apart might

match to different drivers. Redis' native GEO commands (GEOSEARCH) handle this automatically, which is why they're preferred over rolling your own.

8. Quadtrees — The Alternative

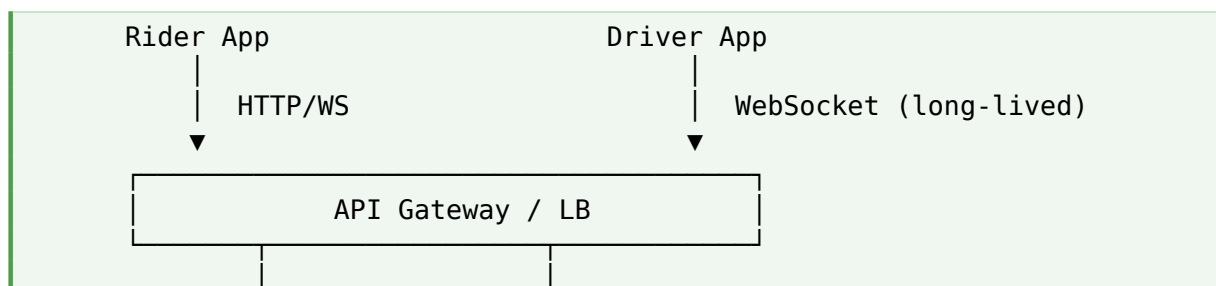
A QUADTREE recursively subdivides a region into 4 quadrants. Each cell splits when it holds too many points. Result: dense areas have small cells (many splits), sparse areas have large cells (few splits) — adaptive to real-world density.

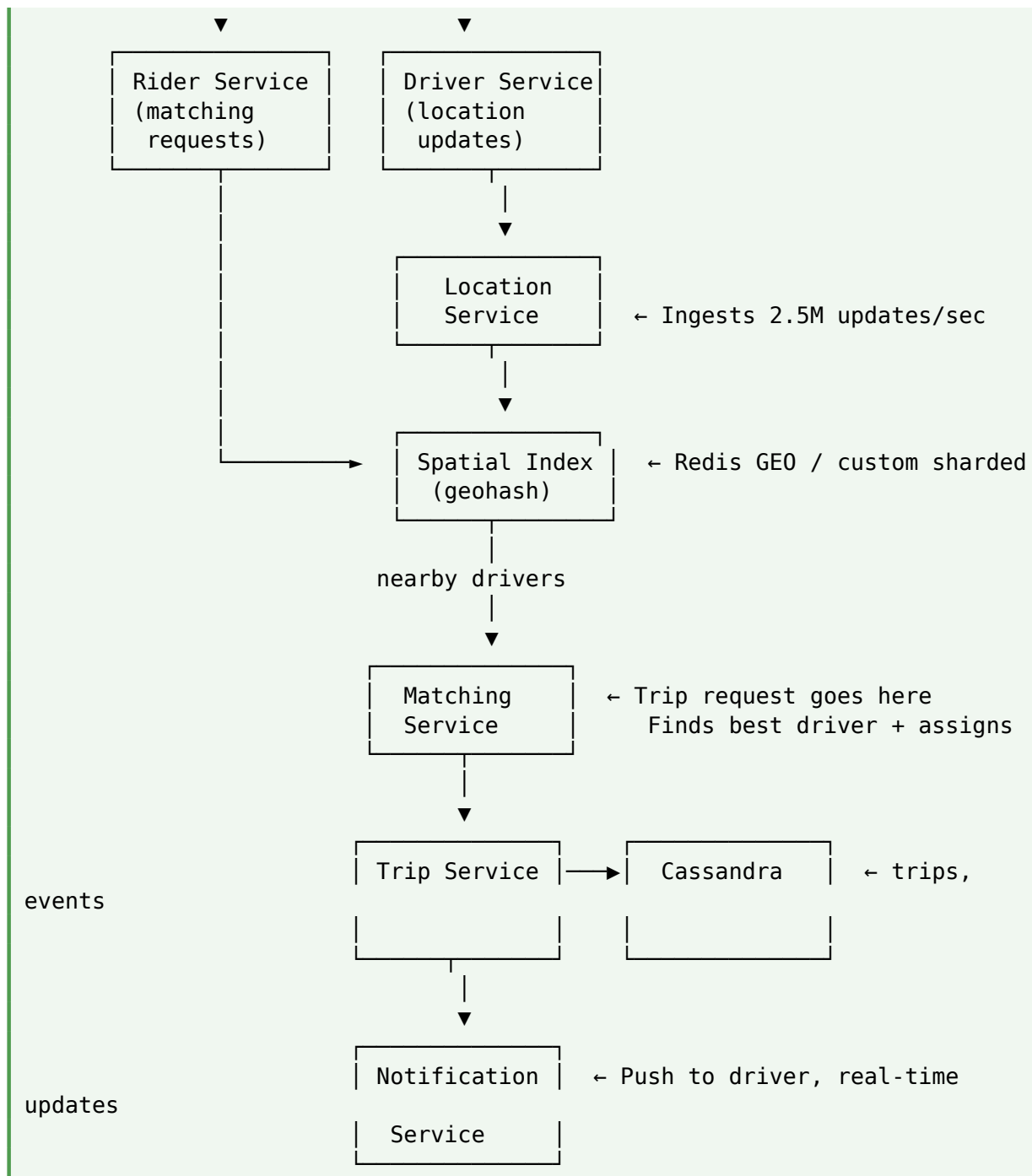
```
Region: entire San Francisco
      |
Split into 4 quadrants (NW, NE, SW, SE)
      |
Any quadrant with > 100 drivers → split again
Sparse quadrants → stay as one big cell
      |
Final: dense downtown has many tiny cells
      |
      outskirts have few large cells
```

Aspect	Geohash	Quadtree
Structure	String prefix hierarchy	Adaptive tree of quadrants
Distribution	Uniform grid worldwide	Denser cells where density is high
Storage	Key-value friendly (Redis, DynamoDB)	Requires tree structure in memory
Range queries	Neighbor cells + prefix search	Traverse tree recursively
Best for	Distributed cache, KV stores	In-memory single-node indexes
Industry use	Uber, Lyft, Google Maps	Some ride-hailing, mapping systems

For interviews: Geohashing is the default answer for ride-hailing — it's what Uber actually uses (or their proprietary H3 hex grid, which is a variant). Mention quadtrees as an alternative + trade-offs. R-trees are common in traditional GIS databases like PostGIS but less used at real-time scale.

9. Step 5 — High-Level Architecture





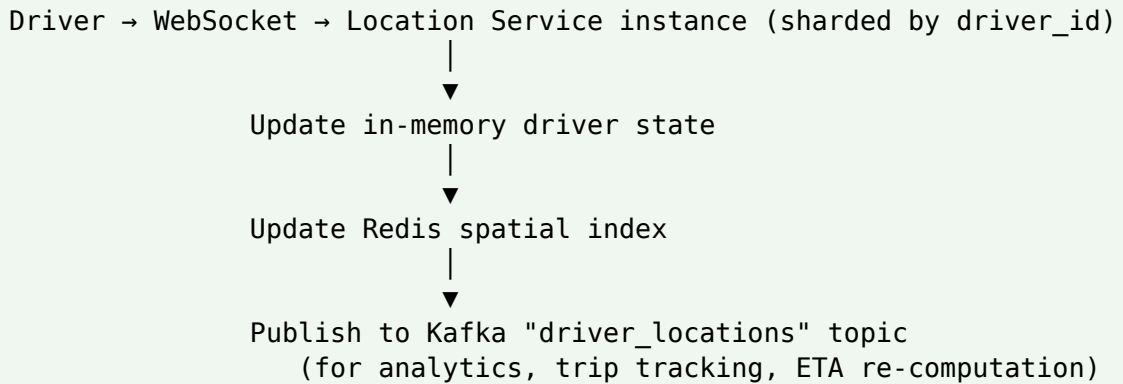
Service Roles

- Location Service — ingests driver location updates, writes to spatial index
- Spatial Index — geohash-keyed store of live driver positions (Redis)
- Matching Service — takes a rider request, queries spatial index, picks best driver
- Trip Service — manages trip lifecycle (requested → matched → started → completed)
- Notification Service — pushes ride offers to drivers, real-time updates to riders

10. Deep Dive: The Location Service

At 2.5M updates/sec, this is the workload that dwarfs everything else. Design decisions here dominate the architecture.

Ingestion Path



Sharding

- Shard by driver_id — each driver's updates go to one node consistently
- Node handles its drivers' updates locally, then propagates to Redis
- At 2.5M updates/sec ÷ 50K/sec/node ≈ 50 Location Service nodes

Don't write every location to Cassandra: That's 2.5M writes/sec forever — unnecessary. Cassandra gets the DELTAS you need (breadcrumbs during a trip, or every N seconds for analytics). Live positions live in Redis with TTL. If a driver stops sending updates, TTL expires and they auto-disappear from the map.

11. Deep Dive: The Matching Service

Matching Algorithm — Straightforward

1. Rider requests ride with pickup location
2. Compute rider's geohash + neighbors
3. Query spatial index → get candidate drivers (say, 30-50 within radius)
4. Filter: available (not on a trip), correct ride type, rating > threshold
5. Rank by (distance, ETA, driver quality, wait time)
6. Send ride offer to top driver via WebSocket
7. If accepted → assign trip. If declined/timeout → offer to next in list

```
public Trip matchDriver(RideRequest req) {  
    List<Driver> candidates = spatialIndex.nearby(req.pickup, radiusKm=5);  
  
    candidates = candidates.stream()
```

```

        .filter(d -> d.available)
        .filter(d -> d.rideType == req.rideType)
        .filter(d -> d.rating >= 4.0)
        .sorted(Comparator.comparingDouble(d -> etaMinutes(d,
req.pickup)))
        .collect(toList());

        for (Driver d : candidates.subList(0, min(5, candidates.size()))) {
            Optional<Response> reply = notifyService.offerRide(d, req,
timeoutSec=15);
            if (reply.isPresent() && reply.get().accepted) {
                return tripService.create(d, req);
            }
        }
        return null;    // no match found
    }
}

```

Race Conditions

The double-booking problem: Two riders in the same area request rides simultaneously. Both see driver D as available. Both try to assign D. Without coordination, D gets "double-booked" — one rider's trip fails at the last moment. Fix: use OPTIMISTIC LOCKING (or Redis SET NX) on the driver's status. Only one write wins; the loser retries with the next candidate.

12. Deep Dive: Trip Lifecycle

```

REQUESTED  (rider tapped "request")
  |
  ▼ Matching Service assigns
MATCHED    (driver notified, has 15 sec to accept)
  |
  ▼ Driver accepts
ACCEPTED   (rider sees driver's info + ETA to pickup)
  |
  ▼ Driver arrives at pickup
ARRIVED
  |
  ▼ Rider gets in, driver starts trip
IN_PROGRESS (real-time location updates to rider)
  |
  ▼ Driver marks trip complete at destination
COMPLETED (fare calculated, payment charged, ratings requested)

```

State Machine

Each state transition is a WRITE to the trips table + an EVENT to trip_events. Both driver and rider apps see updates via WebSocket. The state machine prevents illegal transitions ("can't go from REQUESTED to COMPLETED without ACCEPTED").

Idempotency matters here too: Driver's phone loses connection while marking "arrived." App retries the state change. Without idempotency, arrived_at gets overwritten. Use trip_events append-only log — later events can't corrupt earlier ones. Alternatively: only allow transitions from specific predecessors (server rejects duplicate transitions).

13. Deep Dive: ETA Calculation

"When will my driver arrive?" is one of the most-asked pieces of information. Naive: straight-line distance ÷ average speed. Wrong — doesn't account for one-ways, traffic, or actual road distances. Real solution requires a road graph.

The Approach

- Store the road network as a directed graph (intersections = nodes, road segments = edges)
- Edges weighted by TRAVEL TIME (not distance) — traffic changes this constantly
- Traffic estimates come from historical patterns + live GPS pings from all drivers
- ETA computation: shortest path (Dijkstra, A*, or CONTRACTION HIERARCHIES for speed)

```
// Simplified ETA service
double eta(Location from, Location to) {
    Node fromNode = nearestIntersection(from);
    Node toNode = nearestIntersection(to);
    return roadGraph.shortestPathTime(fromNode, toNode);
}
```

Scaling ETA

Computing shortest paths in real-time is expensive. At scale, use CONTRACTION HIERARCHIES (preprocess the graph to add shortcut edges) or approximate ETAs with a MACHINE LEARNING model trained on historical trip times. Real Uber uses both — a road-network engine for accuracy plus ML for fast approximations.

Never compute exact ETA on every location update: If you recompute ETA every 4 seconds for every trip, you're doing 250K shortest-path queries/sec. Cache the ETA and only recompute on significant driver movement (e.g., > 100 m) OR every 30 seconds. Rider sees a smooth countdown; server does 10× less work.

14. Deep Dive: Surge Pricing

When demand exceeds supply in an area, price multiplier increases to incentivize more drivers to come. It's a market-clearing mechanism — the model itself is well-understood; the interesting engineering is COMPUTING and PROPAGATING the multiplier in near-real-time.

Design

- Divide the city into geohash cells (~1 km precision)

- For each cell, track: active drivers, pending requests, recent decline rate
- Compute surge multiplier every ~30 seconds per cell (demand/supply ratio)
- Broadcast surge levels to all rider apps every ~1 min (via push or poll)
- Rider requests include the surge multiplier at request time — locked in

Surge as a fairness mechanism: When the interviewer asks about surge, mention BOTH the pricing model (economics) AND the engineering (computed per geohash cell, broadcast eventually consistent). Understanding both sides shows senior thinking. Real Uber's surge is much more complex, incorporating ML predictions of demand.

15. Additional Features

15.1 Payments

Fare charged post-trip. Integrate with Stripe or an internal payment service. Never process cards on your servers directly — use tokenization. Payment is a SYNCHRONOUS step: fare compute → charge card → success → complete trip. Failures require retry logic + fallback to alternate payment method.

15.2 Ratings

After trip, both rider and driver rate each other 1-5. Aggregate ratings on both sides. Low-rated drivers can be temporarily disabled or removed. Low-rated riders may find matching harder. Simple design: append-only ratings table, denormalized average on driver/rider records.

15.3 Trip Sharing

"Share ETA with contacts" — generate a short URL (Ch 51 pattern!), embed real-time driver location. Anyone with the URL can view the trip in near-real-time until it completes. Reuse the URL shortener design.

15.4 UberPool / Carpool

Match riders heading in the same direction. Optimization problem — driver picks up rider 1, then rider 2 en route, drops off rider 1 first if closer, then rider 2. Requires graph algorithms + heuristics. Handled by a separate specialized service.

16. Failure Modes

Failure	Mitigation
Location Service node down	That shard's drivers can't update. Consistent hashing failover; drivers reconnect to healthy node.
Spatial index (Redis) down	Matching fails temporarily. Fallback: query recent locations from Kafka log or Cassandra breadcrumbs.

Failure	Mitigation
Matching Service overloaded	Ride requests time out. Regionalize — each city/region has its own Matching Service.
Driver goes offline mid-trip	TTL expires from spatial index. Trip Service alerts rider; may reassign or refund.
Payment fails post-trip	Retry logic; fallback to secondary card; if all fails, add to outstanding balance.
Regional outage (e.g., zone AZ-1 down)	Multi-region deployment — nearby healthy region takes over. Data replicated cross-region.

17. Scaling to Global

Uber operates in 70+ countries. Multi-region design becomes essential:

- REGIONAL DEPLOYMENTS — each city or metro area is its own stack. San Francisco data stays in San Francisco region.
- Users are typically LOCAL to a region → 99% of traffic is intra-region → no cross-region latency
- Cross-region traffic only for user account, payment history — sync via async replication
- Global services (auth, payment) served from nearest region

The locality property: Ride-hailing has strong LOCALITY — a rider in Mumbai never needs to be matched to a driver in Berlin. This makes regional sharding EASY. Compare to Twitter (Ch 52), where a user in Mumbai might follow accounts globally. The design implications are huge — Uber can scale regions independently, while Twitter can't.

18. Trade-offs Summary

Decision	Trade-off
Geohash vs quadtree	Geohash = distributed-friendly, uniform; quadtree = adaptive to density
Update every driver's location vs delta	Every update = fresh map, high infra cost; delta = efficient but potentially stale
Server-side vs client-side ETA	Server = accurate + centralized model; client = simpler + less load
Assign nearest driver vs best fit	Nearest = fast; best fit = considers rating, wait time, direction
Match then confirm vs pre-commit	Match-confirm = simple but slower; pre-commit = fast but complex rollback
Regional shards vs global	Regional = locality wins; global = simpler ops but latency

Decision	Trade-off
Live location in Redis vs DB	Redis = fast; DB = durable but slow for millions of writes/sec

19. Common Mistakes

Mistake 1: Not mentioning spatial indexing

Saying "we search for nearby drivers" without explaining HOW. A linear $O(N)$ scan is a red flag. Geohashing (or quadtree) is the interview-expected answer. Discuss the trade-offs.

Mistake 2: Persisting every location update

Writing 2.5M rows/sec to a durable database is unnecessary and infeasible. Live state in Redis; only trip breadcrumbs go to Cassandra (and only during active trips).

Mistake 3: Ignoring the double-booking race

Two riders can't both claim the same driver. Optimistic locking / atomic status flip on the driver. Miss this and your system happily double-books.

Mistake 4: Straight-line ETAs

ETAs from Euclidean distance are wrong — roads don't go through buildings. Mention the road network graph and ML models for realistic ETAs. Even a hand-wavy "we'd use Google Maps' routing engine" is fine to defer the detail.

Mistake 5: Global architecture for a regional problem

Ride-hailing has strong locality — users rarely cross-region. Push regional deployments early. Miss this and you overengineer for global consistency that isn't needed.

Mistake 6: Not addressing surge as a system concern

Surge isn't just economics — it's a per-cell computation that must be broadcast to millions of rider apps every minute. Design it as a lightweight geohash-keyed pub/sub.

Mistake 7: Forgetting the trip state machine

Trips have 6+ states. State transitions must be validated + logged. Skipping this leads to bugs like "driver marked trip complete but rider never got in."

20. Best Practices

- Use geohashing (or Redis GEO commands) for spatial indexing — it's the standard
- Always query neighbor cells too — border effects matter
- Live driver state in Redis with TTL — auto-cleanup on disconnect

- Optimistic locking on driver.status to prevent double-booking
- Regional deployments — cities are largely independent
- Trip state machine with event log — safest way to track state changes
- Cache ETAs, don't recompute on every location update
- Surge computed per geohash cell, broadcast eventually consistent
- Idempotent trip state transitions — reject duplicates server-side

21. Quick Reference

Concept	Meaning
Geohashing	Encode lat/lng as short string; neighbors share prefix
Quadtree	Recursively subdivide 2D space; adaptive to density
Spatial index	Data structure for efficient 'nearby X' queries
Redis GEO	Native geohash commands (GEOADD, GEOSEARCH)
Border effect	Nearest might be in a neighboring cell; always query 8 neighbors
Trip state machine	REQUESTED → MATCHED → ACCEPTED → ARRIVED → IN_PROGRESS → COMPLETED
Double-booking	Race condition — two riders claiming same driver; fix with locking
Optimistic locking	Compare-and-swap on driver.status; retry on conflict
ETA road graph	Intersections=nodes, road segments=edges weighted by travel time
Contraction hierarchies	Preprocessed shortcuts for fast shortest-path queries
Surge pricing	Multiplier per geohash cell based on demand/supply
Regional deployment	Each city/metro has its own stack — locality wins
Location TTL	Auto-expire driver locations after 60s of no updates

22. Related Interview Problems

The location-based patterns from Uber apply to many other systems:

8. Design DoorDash / Uber Eats — restaurants (fixed) + riders + drivers (moving), same core
9. Design Yelp / Restaurant search — no drivers, but same 'find nearby X' problem
10. Design a Dating App (Tinder) — 'nearby users' matching, same spatial index
11. Design AirTag / Find My iPhone — offline peer-to-peer proximity + crowd-sourced location

12. Design a Traffic / Navigation App (Waze) — real-time road-graph updates, ETA computation
13. Design a Fleet Management System — vehicles-only version of Uber, no matching
14. Design a Bike/Scooter Sharing (Lime) — very similar to Uber but with static parked vehicles
15. Design a Real-Estate Search (Zillow) — spatial queries with property filters

23. Final Takeaway

Uber's design is fundamentally different from feeds and chats because of one thing: SPATIAL INDEXING. Geohashing is the most important concept in this chapter — internalize it and you can design any location-based system. Beyond the spatial index, Uber layers in real-time matching, trip state machines, and ETA calculation, but the geohash is what makes 'find 20 drivers near this rider in <100ms' possible.

For interviews, memorize the flow: capacity (2.5M location updates/sec = biggest workload) → spatial index (geohash + neighbors) → Location Service (ingests, updates Redis) → Matching Service (queries index, ranks, offers ride, handles decline) → Trip Service (state machine, event log) → regional deployment (locality wins). The double-booking race, border effect, and cached ETAs are the operational details interviewers dig into.

Key Takeaway: Uber = spatial matching problem. GEOHASHING encodes lat/lng as strings where neighbors share prefixes — enables $O(\log n)$ 'nearby X' queries. QUADTREES are adaptive alternatives. Live driver locations in REDIS with TTL (2.5M updates/sec is too much for durable DB). MATCHING SERVICE queries spatial index, ranks candidates, offers ride via WebSocket with optimistic locking to prevent DOUBLE-BOOKING. TRIP STATE MACHINE tracks 6 states with append-only event log. ETAs from a ROAD GRAPH, cached to avoid recomputation. REGIONAL DEPLOYMENTS — locality is your friend.

24. What's Next?

Four canonical designs done (URL shortener, feed, chat, Uber). Remaining directions:

- Design YouTube — video encoding pipeline, adaptive bitrate, CDN edge caching
- Design a Rate Limiter — token bucket at distributed scale
- Design a Distributed Cache — Redis internals, consistent hashing, replication
- Design a Notification System — cross-channel delivery, priorities, deduplication
- Design a Search Autocomplete — trie-based, prefix caching, popularity scoring
- Design a Payment System — idempotency, sagas, PCI compliance
- Deep Dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep Dive: Kafka Architecture — partitions, consumer groups, exactly-once semantics